

Dokumentation React-Projekt

learn-IDI Hub

Kurs:	Skript- und Webtechnologien
Studiengang:	Informatik: Digital Innovation
Name:	Kohler Robin-Pascal (korp)
Datum:	18.01.2026

1. Die Vision

Das Projekt **learn-IDI** ist eine Web-Applikation, die als zentraler Hub für mein Tutorium im Studiengang **Informatik: Digital Innovation (IDI)** dient. Zielgruppe sind die Erst- und Zweitsemestrigen, die im Rahmen des Tutoriums regelmäßig Unterlagen, Aufgaben, Code-Beispiele und organisatorische Infos benötigen. Die Anwendung bündelt dafür mehrere Funktionen: einen übersichtlichen Timetable mit schneller „Nächste Einheit“-Übersicht, eine Unterlagen-Seite, auf der Dateien je Lehrveranstaltung strukturiert zum Ansehen und Download verfügbar sind, und eine Page für externe hilfreiche Ressourcen mitsamt Filter-Möglichkeiten. Damit entsteht eine Plattform, die sowohl organisatorische als auch inhaltliche Unterstützung vereint.

Das Projekt löst das Problem, dass das Tutorium bisher über mehrere Kanäle verteilt ist: Kommunikation läuft hauptsächlich über MS Teams und gelegentlich E-Mail, während Dokumente (zB. Timetable, Aufgaben, Beispiele) auf OneDrive liegen und als Ordnerstruktur geteilt werden. Diese Trennung macht es für die Studierende oft mühsam, schnell das Richtige zu finden. Und für mich bedeutet sie laufenden Pflegeaufwand und potenziell Verwirrung durch verschiedene Linkstände oder Versionen. **learn-IDI** schafft hier Abhilfe, indem es die wichtigsten Tutoriums-Inhalte an einem Ort zusammenführt, klar strukturiert präsentiert und den Zugriff vereinheitlicht.

Statt einzelne Inhalte immer wieder in Teams-Chats zu pinnen, Ordnerlinks nachzureichen oder Dateien zu aktualisieren, wollte ich eine Lösung, die sich wie ein „kleines Portal“ anfühlt: schnell, übersichtlich, nutzerfreundlich und so praktisch, dass Studierende es tatsächlich gerne verwenden.

Mein Ziel war es, eine stabile und wartungsarme Plattform zu bauen, die (1) für Studierende sofort Mehrwert bietet und (2) für mich als Tutor unkompliziert zu pflegen ist. Dazu gehören Features wie thematisch aufgeteilte Unterlagen pro Lehrveranstaltung, automatische Index-Generierung der Dateien aus dem Repository, Preview-Funktionen für `.md` und `.sql`, sowie Filter- und Suchfunktionen zur schnellen Navigation. Der Erfolg des Projekts misst sich daran, ob die Plattform im Tutoriumsalltag wirklich genutzt wird. Wenn **learn-IDI** als zuverlässiger „Single Point of Truth“ für das Tutorium funktioniert und die Hürde zum Nachschlagen und Vorbereiten senkt, hat das Projekt sein Ziel erreicht.

2. Architektur & Implementierung

2.1 Grundsätze

Bei der Umsetzung von **learn-IDI** war es mir besonders wichtig, von Beginn an auf eine saubere, konsistente und langfristig wartbare Architektur zu achten. Da ich das Projekt potenziell noch erweitere, um es auch wirklich im Tutorium einzusetzen, sollte der Code nicht nur funktionieren, sondern auch gut lesbar, logisch strukturiert und leicht erweiterbar sein. Im Fokus standen dabei eine klare Ordnerstruktur, eine strikte Trennung von Zuständigkeiten sowie nachvollziehbare Styling-Konzepte. Die folgenden Unterpunkte beschreiben die wichtigsten architektonischen Prinzipien, die ich im Projekt konsequent eingehalten habe.

2.1.1 Seitenbasierte Ordnerstruktur

Jede Seite der Anwendung besitzt einen eigenen Unterordner innerhalb des `pages`-Verzeichnisses. Dieser Ordner enthält stets eine `index.js`, welche die eigentliche Seitenkomponente (z. B. `Page.jsx`) exportiert. Dadurch kann eine Seite beim Import stets über `pages/PageName` eingebunden werden, ohne den vollständigen Pfad zur eigentlichen JSX-Datei angeben zu müssen. Dies reduziert Redundanz beim Importieren und sorgt für ein einheitliches, gut lesbares Import-Schema im gesamten Projekt.

Zusätzlich besitzt jede Seite einen eigenen Unterordner `components`, in dem ausschließlich jene React-Komponenten liegen, die nur von dieser Seite verwendet werden. Diese Komponenten werden bewusst nicht global abgelegt, um eine klare Trennung zwischen seitenlokaler und seitenübergreifender Funktionalität zu gewährleisten. Innerhalb dieses `components`-Ordners habe ich auf weitere Verschachtelungen verzichtet, da mir eine möglichst flache Ordnerstruktur wichtig war. Dadurch bleibt die Navigation im Projekt übersichtlich und die Zugehörigkeit einer Komponente zu einer Seite ist jederzeit eindeutig erkennbar.

2.1.2 Styling mit CSS Modules

Für das Styling einzelner Komponenten habe ich CSS Modules verwendet, unterstützt von ein paar wenigen, global definierten Utility Klassen. Jede `.jsx`-Datei besitzt eine gleichnamige `.module.css`-Datei, in der ausschließlich die Styles dieser Komponente definiert sind. Der Hauptgrund für diese Entscheidung ist die automatische Kapselung von CSS-Klassen: Klassennamen sind lokal scoped und können sich somit nicht unbeabsichtigt gegenseitig überschreiben. Dadurch entfallen viele typische Probleme von globalem CSS, insbesondere bei wachsenden Projekten. Zusätzlich erleichtert dieser Ansatz das Refactoring und Wiederverwenden von Komponenten, da Styles und Logik immer gemeinsam betrachtet werden können.

2.1.3 Globales CSS-Konzept mit Layern

Neben den komponentenspezifischen Styles existiert eine zentrale `global.css`, die das grundlegende Styling der Anwendung definiert. Diese Datei ist in klar strukturierte CSS-Layer unterteilt, die in folgender Reihenfolge aufgebaut sind:

`reset → base-tokens → themes → derived-tokens → base → components → utilities → overrides`

Jeder Layer erfüllt eine klar definierte Aufgabe. Design-Tokens, Farbvariablen und Themes sind strikt von komponentenspezifischem Styling getrennt, während Utility-Klassen und optionale Overrides

bewusst an das Ende der Kaskade gelegt sind. Sämtliches CSS der Anwendung befindet sich immer in dem jeweils semantisch passenden Layer und niemals außerhalb dieser Struktur.

Der Vorteil dieses Ansatzes liegt in mehreren Bereichen: Zum einen entstehen praktisch keine Specificity-Probleme, da die Reihenfolge und Verantwortung der Styles klar geregelt ist. Zum anderen erleichtert dies das Debugging erheblich, da moderne Browser in den DevTools den jeweiligen CSS-Layer anzeigen und somit sofort ersichtlich ist, aus welchem Kontext eine Regel stammt. Nicht zuletzt sorgt diese Struktur auch im Code selbst für Übersichtlichkeit, insbesondere da sich das Styling auf mehrere Dateien (Global Styles, Theme-Dateien und CSS Modules) verteilt.

2.2 Datenfluss & State Handling

Der Datenfluss in **learn-IDI** basiert vollständig auf statischen JSON-Dateien, die zur Laufzeit im Frontend geladen werden. Beispiele dafür sind der automatisch generierte Index der Unterlagen (`materials.index.json`) sowie weitere Konfigurations- und Inhaltsdateien. Die Daten werden über einfache `fetch`-Aufrufe geladen und anschließend im lokalen React-State verarbeitet (Anzeigen, Filtern, Sortieren, ...).

Dieser Ansatz ist bewusst gewählt, da er für ein statisch gehostetes Projekt sehr einfach, performant und wartungsarm ist. Gleichzeitig ist die Architektur so aufgebaut, dass die JSON-Dateien jederzeit ohne größeren Umbau durch API-Endpunkte ersetzt werden könnten, da sich der Zugriff im Frontend bereits an einem klar definierten Datenmodell orientiert. Dadurch bleibt das Projekt auch für eine spätere Erweiterung mit einem Backend offen.

2.3 Nutzung von KI

Künstliche Intelligenz wurde in diesem Projekt gezielt als unterstützendes Werkzeug eingesetzt. Insbesondere das Styling der einzelnen Komponenten wurde mithilfe von ChatGPT entworfen und anschließend von mir leicht angepasst. Dadurch konnte ich mich stärker auf Struktur, User Experience und Konsistenz konzentrieren, ohne jedes UI-Detail manuell von Grund auf entwerfen zu müssen.

Darüber hinaus habe ich ChatGPT zur Entwicklung der Logik in JavaScript genutzt, vor allem bei den Hilfsfunktionen in `/utils/time.js` zur Berechnung von Zeitfenstern, relativen Zeitangaben und Datumsvergleichen.

Ein weiterer wichtiger Einsatzbereich war die konzeptionelle Phase des Projekts: Ideen, Layouts und Seitenstrukturen wurden gemeinsam diskutiert, Vor- und Nachteile abgewogen und anschließend in eine konkrete Implementierung überführt. Zusätzlich diente KI als Nachschlage- und Lernhilfe für die verwendeten Technologien, etwa bei Best Practices, Architekturentscheidungen und beim Debugging. Auch hier in der Dokumentation findet Künstliche Intelligenz sinnvolle Verwendung, um Texte zu überprüfen, mögliche Fehler zu korrigieren sowie den Lesefluss und die Verständlichkeit gezielt zu verbessern.

3. Reflexion mit Blick in die Zukunft

3.1 Probleme, Unsicherheiten & offene Fragen

Da **learn-IDI** mein erstes Projekt mit React ist, gab es während der Entwicklung einige Entscheidungen, bei denen ich unsicher war, ob sie gängigen Best Practices entsprechen oder eher aus meiner eigenen Intuition heraus entstanden sind. Ein Beispiel dafür ist der Umgang mit Icons. SVG-Icons, die ein fester Bestandteil der Website sind, habe ich gesammelt in einer zentralen JSX-Datei (`/src/components/Icons.jsx`) abgelegt und importiere sie von dort als Komponenten, wo sie benötigt werden. Icons, die als Logos für externe Ressourcen dienen, liegen hingegen als separate `.svg`-Dateien vor. Dieser Ansatz erscheint mir logisch und übersichtlich, da zwischen statischen UI-Icons und austauschbaren Ressourcen klar unterschieden wird. Gleichzeitig bleibt offen, ob ein erfahrener React-Entwickler diesen Weg ebenfalls wählen würde oder stattdessen auf Icon-Libraries, dynamische Imports oder ausschließlich statische Assets setzen würde.

Ähnliche Unsicherheiten gab es bei der Wiederverwendung von UI-Komponenten. Rückblickend stellt sich die Frage, ob es sinnvoll gewesen wäre, mehr allgemeine, seitenübergreifend genutzte UI-Komponenten (zB. Cards, Filterleisten, ...) in einem gemeinsamen Verzeichnis wie `src/components/ui` zu bündeln, anstatt bestimmte Layout- und Styling-Entscheidungen stärker in den jeweiligen CSS Modulen der einzelnen Seiten zu belassen. Zwar sorgt der aktuelle Ansatz für eine sehr klare Zuordnung von Logik und Styling, allerdings könnte ein stärker abstrahiertes UI-Layer die Wiederverwendbarkeit und Konsistenz bei weiterem Projektwachstum noch erhöhen.

3.2 Meine Lernerkenntnisse

Vor diesem Projekt habe ich Websites hauptsächlich entweder mit klassischem HTML, CSS und JavaScript in Kombination mit PHP oder Node.js umgesetzt oder auf bestehende Content-Management-Systeme wie WordPress zurückgegriffen. Im Vergleich dazu stellt das Arbeiten mit React einen deutlich anderen Denkansatz dar. Statt Seiten als statische Dokumente zu betrachten, steht das Zerlegen der Oberfläche in wiederverwendbare, zustandsabhängige Komponenten im Vordergrund.

Besonders lehrreich war für mich der Umgang mit State und abgeleiteten Daten. Die Arbeit mit statischen JSON-Dateien und automatisch generierten Indizes hat mir gezeigt, wie gut sich React für datengetriebene Oberflächen eignet, selbst ohne den Einsatz eines klassischen Backends. Gleichzeitig habe ich ein besseres Verständnis für moderne Frontend-Architektur und komponentenbasiertes Arbeiten gewonnen, das mir langfristig mehr Kontrolle und Skalierbarkeit ermöglicht.

3.3 Offenheit für zukünftige Erweiterungen

Einige Implementierungsdetails habe ich bewusst so gestaltet, dass sie Raum für zukünftige Weiterentwicklung lassen. Ein Beispiel dafür ist der CSS-Layer `overrides`, der aktuell noch nicht aktiv genutzt wird, aber gezielt für spätere Anpassungen vorgesehen ist, ohne bestehende Styles umbauen oder überschreiben zu müssen.

Auch funktional ist das Projekt offen konzipiert. Der Einsatz von JSON-Dateien anstelle einer API vereinfacht zwar den aktuellen Betrieb, lässt jedoch eine spätere Backend-Anbindung problemlos zu. **learn-IDI** ist somit nicht nur ein abgeschlossenes Projekt, sondern auch eine solide Basis für mögliche Erweiterungen.